НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
Факультет прикладної математики
Кафедра прикладної математики

Звіт

із Лабораторної роботи №1

на тему «Інтерполяційні поліноми»

Варіант №10

із дисципліни «Чисельні методи»

Виконав:
студент групи КМ-32
Мурсюкаєв Д. А.

Керівник:
доцент Андрусенко О. М.

Київ – 2025

ЗМІСТ

ЗАВДАННЯ 3

ІНТЕРПОЛЯЦІЙНИЙ МНОГОЧЛЕН ЛАГРАНЖА 4

ІНТЕРПОЛЯЦІЙНИЙ МНОГОЧЛЕН НЬЮТОНА 5

АНАЛІТИЧНИЙ ВИГЛЯД ФУНКЦІЙ 6

ГРАФІКИ ФУНКЦІЙ 7

ЗНАЧЕННЯ ДЛЯ ДОВІЛЬНИХ ТОЧОК 9

ДОДАТОК А 10

ЗАВДАННЯ

Задана функція: $\lg(3x-2)$

Обмеження для функції: $x > \frac{2}{3}$

На довільному проміжку $[a;b]$ зі сталим кроком h розглянути 5 точок – вузлів інтерполяції:

$$a = x_0 < x_1 < \ldots < x_4 = b$$

Побудувати:

- Многочлен Лагранжа $\mathrm{L}_n(x)$;
- Многочлен Ньютона $\mathrm{N}_n(x)$.

На одному графіку побудувати:

- $y = f(x)$;
- $y = \mathrm{L}_n(x)$;
- $y = \mathrm{N}_n(x)$.

і зазначити вузли інтерполяції.

Вибрати:

- Довільну точку x^1 з проміжку $[a;b]$ таку, що $x^1 \neq x_i$;
- Довільну точку $x^2 : x^2 < a$.

Обчислити:

- $f(x^1)$, $\mathrm{L}_n(x^1)$, $\mathrm{N}_n(x^1)$;
- $f(x^2)$, $\mathrm{L}_n(x^2)$, $\mathrm{N}_n(x^2)$

ІНТЕРПОЛЯЦІЙНИЙ МНОГОЧЛЕН ЛАГРАНЖА

Обраний проміжок [10; 90] зі сталим кроком 20.

Вузли інтерполяції: 10, 30, 50, 70, 90.

Задана функція: $f(x) = \lg(3x - 2)$

Формули для побудови многочлена Лагранжа:

$$y_i = f(x_i) \tag{1.1}$$

$$c_i = \frac{1}{(x_i - x_0)(x_i - x_1)\dots(x_i - x_{i-1})(x_i - x_{i+1})\dots(x_i - x_n)} \tag{1.2}$$

$$p_i(x) = c_i \cdot (x - x_0)(x - x_1)\dots(x - x_{i-1})(x - x_{i+1})\dots(x - x_n) \tag{1.3}$$

$$\mathrm{L}_n(x) = \sum_{i=0}^{n} p_i(x) \cdot y_i \tag{1.4}$$

ІНТЕРПОЛЯЦІЙНИЙ МНОГОЧЛЕН НЬЮТОНА

Формули для побудови многочлена Ньютона зі сталим кроком h (перша інтерполяційна формула Ньютона):

$$y_n = f(x_n) \tag{2.1}$$

$$\Delta y_{n-1} = y_n - y_{n-1} \tag{2.2}$$

$$\Delta^k y_{n-1} = \Delta^{k-1} y_n - \Delta^{k-1} y_{n-1} \tag{2.3}$$

$$t = \frac{x - x_0}{h} \tag{2.4}$$

$$\begin{aligned} \mathrm{P}_n(x_0 + ht) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_0 + \frac{t(t-1)(t-2)}{3!}\Delta^3 y_0 + \ldots + \\ + \frac{t(t-1)\ldots(t-n+1)}{n!}\Delta^n y_0 \end{aligned} \tag{2.5}$$

АНАЛІТИЧНИЙ ВИГЛЯД ФУНКЦІЙ

За допомогою виконання коду, наведеному у Додатку А, отримаємо наступні значення:

Таблиця 1

x	$f(x)$	$\mathrm{L}_n(x)$	$\mathrm{N}_n(x)$
10	1.447	1.4472	1.4472
20	1.763	1.7479	1.7479
30	1.944	1.9445	1.9445
40	2.072	2.0756	2.0756
50	2.17	2.1703	2.1703
60	2.25	2.2478	2.2478
70	2.318	2.3181	2.3181
80	2.377	2.3813	2.3813
90	2.428	2.4281	2.4281
100	2.474	2.4397	2.4397

Як можна побачити, значення $\mathrm{L}_n(x)$ та $\mathrm{N}_n(x)$ для одного x співпадають, оскільки вони мають однакові вузли інтерполяції (виділені у табл. 1). Отже, за допомогою вбудованих методів середовища Python можемо побудувати аналітичний вигляд функцій многочленів Лагранжа та Ньютона, причому, вони будуть еквівалентні:

$$\mathrm{L}_n(x) = -3.993 \cdot 10^{-8}x^4 + 1.042 \cdot 10^{-5}x^3 - 0.001046x^2 + 0.05475x + 0.9943$$
$$\mathrm{N}_n(x) = -3.993 \cdot 10^{-8}x^4 + 1.042 \cdot 10^{-5}x^3 - 0.001046x^2 + 0.05475x + 0.9943$$

ГРАФІКИ ФУНКЦІЙ

На рис. 1 зображено графік значень $f(x)$, $L_n(x)$, $N_n(x)$ та виділені вузли інтерполяції для проміжку [10; 100] зі сталим кроком 20.

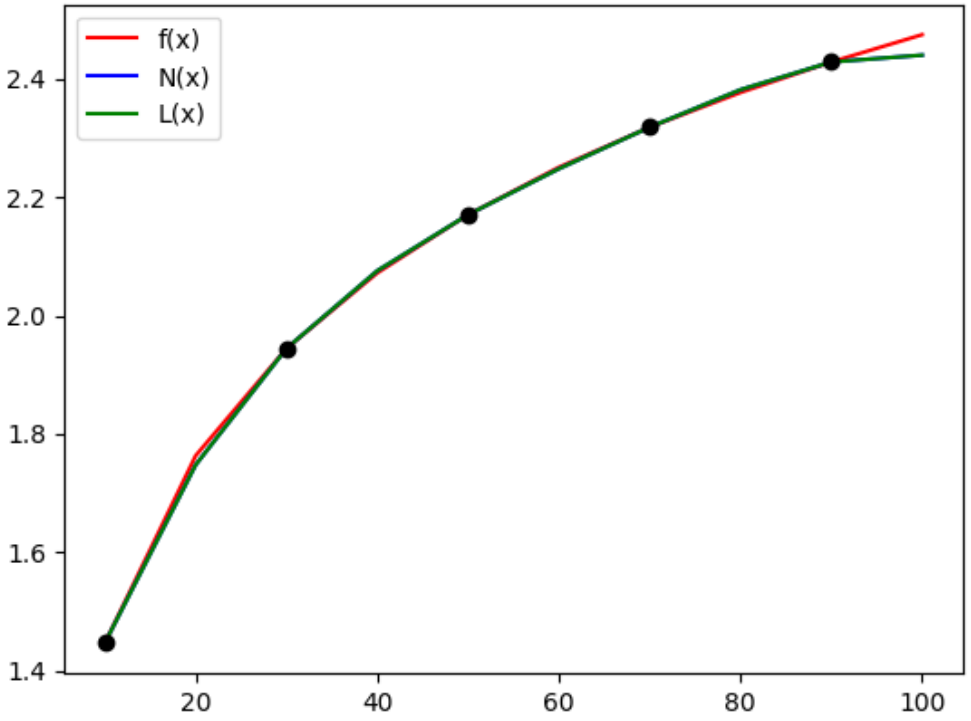


Рис.1

На рис. 2 зображено гладкий графік значень $f(x)$, $L_n(x)$, $N_n(x)$ та виділені вузли інтерполяції для проміжку [10; 100] зі сталим кроком 20.

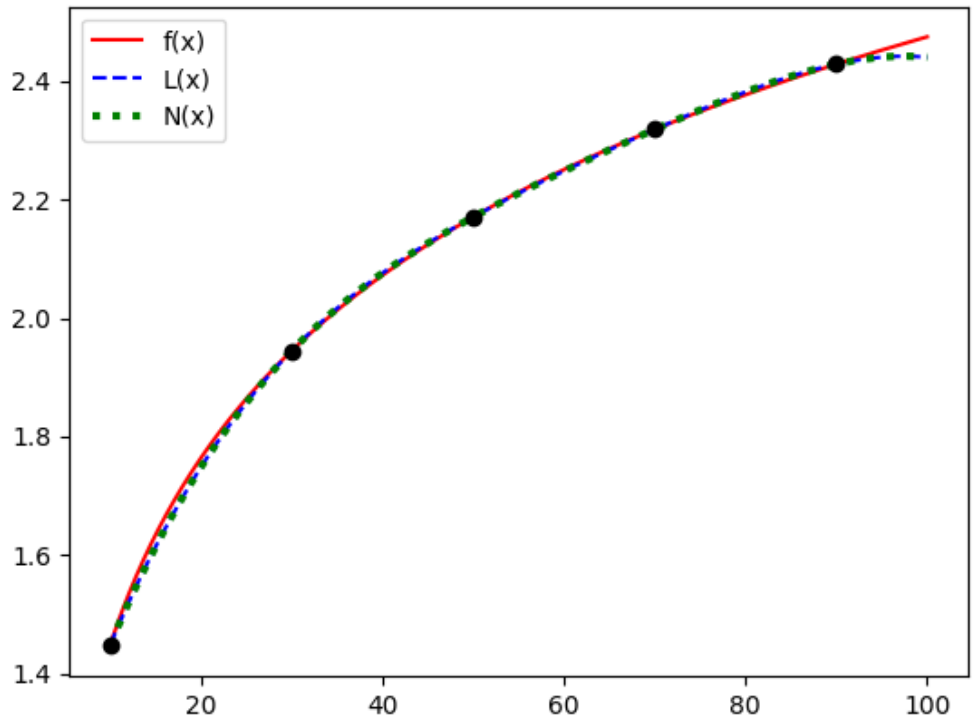


Рис. 2

На рис. 3 зображено гладкий графік значень $f(x)$, $L_n(x)$, $N_n(x)$ та виділені вузли інтерполяції для проміжку $[1; 200]$ зі сталим кроком 20.

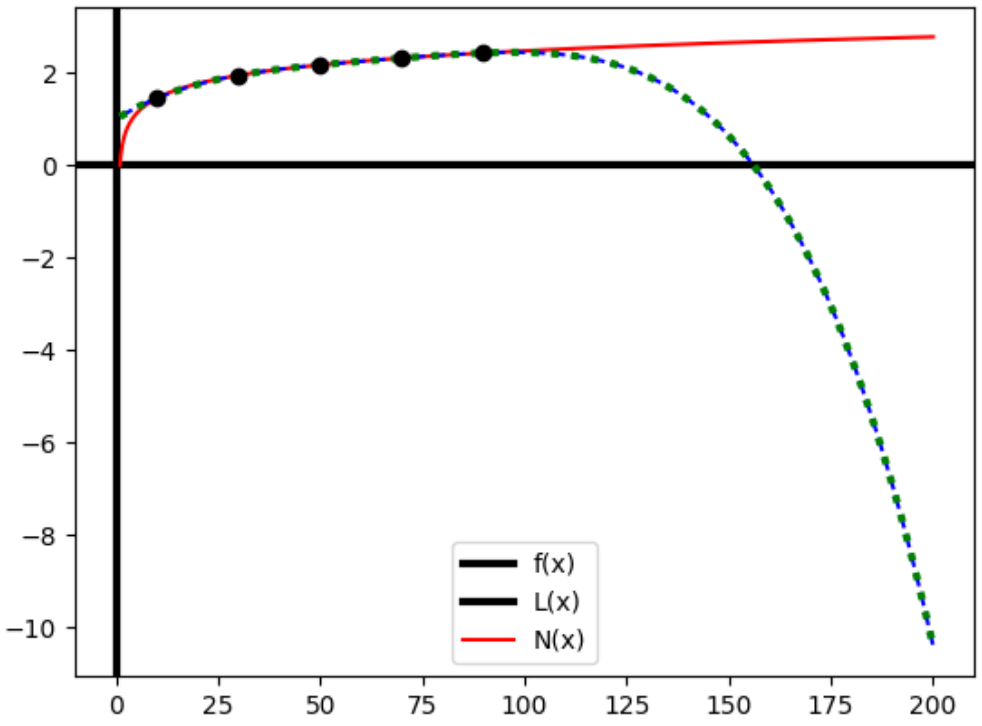


Рис. 3

ЗНАЧЕННЯ ДЛЯ ДОВІЛЬНИХ ТОЧОК

Обрані 2 довільні точки:

- $x^1 = 74$;
- $x^2 = 3$.

Значення $f(x)$, $\mathrm{L}_n(x)$, $\mathrm{N}_n(x)$ для точок x^1 та x^2 наведені у табл. 2.

Таблиця 2

x	$f(x)$	$\mathrm{L}_n(x)$	$\mathrm{N}_n(x)$
74	2.3424	2.3445	2.3445
3	0.8451	1.1494	1.1494

ДОДАТОК А

Лістинг програми:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import lagrange
from scipy.interpolate import KroghInterpolator as newton


x = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

y = [0,0,0,0,0,0,0,0,0,0]
for i in range(10):
    y[i] = (np.log10(3*x[i] - 2))

c = [0,0,0,0,0,0,0,0,0,0]
c[0] = 1 / ((x[0] - x[2]) * (x[0] - x[4]) * (x[0] - x[6]) * (x[0] - x[8]))
c[2] = 1 / ((x[2] - x[0]) * (x[2] - x[4]) * (x[2] - x[6]) * (x[2] - x[8]))
c[4] = 1 / ((x[4] - x[0]) * (x[4] - x[2]) * (x[4] - x[6]) * (x[4] - x[8]))
c[6] = 1 / ((x[6] - x[0]) * (x[6] - x[2]) * (x[6] - x[4]) * (x[6] - x[8]))
c[8] = 1 / ((x[8] - x[0]) * (x[8] - x[2]) * (x[8] - x[4]) * (x[8] - x[6]))


yc = [0,0,0,0,0,0,0,0,0,0]
for i in range(10):
    yc[i] = c[i] * y[i]

l0 = [0,0,0,0,0,0,0,0,0,0]

for i in range(10):
    l0[i] = yc[0] * (x[i] - x[2]) * (x[i] - x[4]) * (x[i] - x[6]) * (x[i] -
x[8])

l1 = [0,0,0,0,0,0,0,0,0,0]

for i in range(10):
    l1[i] = yc[2] * (x[i] - x[0]) * (x[i] - x[4]) * (x[i] - x[6]) * (x[i] -
x[8])

l2 = [0,0,0,0,0,0,0,0,0,0]

for i in range(10):
    l2[i] = yc[4] * (x[i] - x[0]) * (x[i] - x[2]) * (x[i] - x[6]) * (x[i] -
x[8])

l3 = [0,0,0,0,0,0,0,0,0,0]

for i in range(10):
    l3[i] = yc[6] * (x[i] - x[0]) * (x[i] - x[2]) * (x[i] - x[4]) * (x[i] -
x[8])

l4 = [0,0,0,0,0,0,0,0,0,0]

for i in range(10):
    l4[i] = yc[8] * (x[i] - x[0]) * (x[i] - x[2]) * (x[i] - x[4]) * (x[i] -
x[6])

l = [0,0,0,0,0,0,0,0,0,0]

for i in range(10):
```

```
    l[i] = l0[i] + l1[i] + l2[i] + l3[i] + l4[i]
    print(f"{x[i]:^4}| {y[i].round(3):^7}| {l[i].round(4)}")


h = 20

a = [10, 30, 50, 70, 90]

b = [0,0,0,0,0]

for i in range(5):
    b[i] = np.log10(3*a[i] - 2)

d_1 = [0,0,0,0]

for i in range(4):
    d_1[i] = b[i+1] - b[i]

d_2 = [0,0,0]

for i in range(3):
    d_2[i] = d_1[i+1] - d_1[i]

d_3 = [0,0]

for i in range(2):
    d_3[i] = d_2[i+1] - d_2[i]

d_4 = [0]

d_4[0] = d_3[1] - d_3[0]

a_set = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
b_set = [0,0,0,0,0,0,0,0,0,0]

for i in range(10):
    b_set[i] = np.log10(3*a_set[i] - 2)

t = [0,0,0,0,0,0,0,0,0,0]

a_0 = 10

b_0 = np.log10(3*a_0 - 2)

for i in range(10):
    t[i] = (a_set[i] - a_0) / h

n_0 = [0,0,0,0,0,0,0,0,0,0]

for i in range(10):
    n_0[i] = b_0

n_1 = [0,0,0,0,0,0,0,0,0,0]

for i in range(10):
    n_1[i] = t[i] * d_1[0]

n_2 = [0,0,0,0,0,0,0,0,0,0]

for i in range(10):
    n_2[i] = t[i] * (t[i] - 1) * d_2[0] / 2

n_3 = [0,0,0,0,0,0,0,0,0,0]

for i in range(10):
    n_3[i] = t[i] * (t[i] - 1) * (t[i] - 2) * d_3[0] / 6
```

```
n_4 = [0,0,0,0,0,0,0,0,0,0]

for i in range(10):
    n_4[i] = t[i] * (t[i] - 1) * (t[i] - 2) * (t[i] - 3) * d_4[0] / 24

n_n = [0,0,0,0,0,0,0,0,0,0]
for i in range(10):
    n_n[i] = n_0[i] + n_1[i] + n_2[i] + n_3[i] + n_4[i]

print("------")
for i in range(10):
    print(f"{a_set[i]:^4} | {b_set[i].round(3):^7} | {l[i].round(4):^8} |
{n_n[i].round(4):^8}")

print("------")

Lg = lagrange(a, b)
print("L = ", Lg)

Nt = newton(a, b)
coeffs_n = np.polyfit(a, b, len(a)-1)
Nt_poly = np.poly1d(coeffs_n)

print("------")

print("N = ")
print(Nt_poly)

print("------")
point_1 = 74
point_2 = 3

print(f"{point_1:^4}| {np.log10(3*point_1 - 2).round(4):^7} |
{Lg(point_1).round(4):^7} | {Nt(point_1).round(4):^7}")
print(f"{point_2:^4}| {np.log10(3*point_2 - 2).round(4):^7} |
{Lg(point_2).round(4):^7} | {Nt(point_2).round(4):^7}")

print("------")

plt.plot(x, y, color = 'red')
plt.plot(a_set, n_n, color = 'blue')
plt.plot(a_set, l, color = 'green')
plt.plot(a, b, color = 'none', marker = 'o', markerfacecolor = 'black')
plt.legend(['f(x)', 'N(x)', 'L(x)'])
plt.show()

a_set_s = np.linspace(10, 100, 1000)

b_set_s = []

for i in range(1000):
    b_set_s.append(np.log10(3*a_set_s[i] - 2))

lg_s = []
nt_s = []

for i in range(1000):
    lg_s.append(Lg(a_set_s[i]))
    nt_s.append(Nt(a_set_s[i]))


plt.plot(a_set_s, b_set_s, color = 'red')
plt.plot(a_set_s, lg_s, color = 'blue', linestyle = '--')
plt.plot(a_set_s, nt_s, color = 'green', linestyle = ':', linewidth = 3)
plt.plot(a, b, color = 'none', marker = 'o', markerfacecolor = 'black')
```

```
plt.legend(['f(x)', 'L(x)', 'N(x)'])
plt.show()

big_set = np.linspace(1, 200, 1000)

big_set_s = []

for i in range(1000):
    big_set_s.append(np.log10(3*big_set[i] - 2))

big_lg = []
big_nt = []

for i in range(1000):
    big_lg.append(Lg(big_set[i]))
    big_nt.append(Nt(big_set[i]))

fig, ax = plt.subplots()
ax.axhline(y = 0, color = 'black', linewidth = 3)
ax.axvline(x = 0, color = 'black', linewidth = 3)
plt.plot(big_set, big_set_s, color = 'red')
plt.plot(big_set, big_lg, color = 'blue', linestyle = '--')
plt.plot(big_set, big_nt, color = 'green', linestyle = ':', linewidth = 3)
plt.plot(a, b, color = 'none', marker = 'o', markerfacecolor = 'black')
plt.legend(['f(x)', 'L(x)', 'N(x)'])
plt.show()
```